

Interrupts Case Studies: Debuggers and Apollo Guidance Computer COMS10015

Dr Tom Deakin

University of Bristol

Week 20

Debuggers

Mistakes

- ▶ When programmers write programs, they will probably make some mistakes.
- ▶ Mistakes might show up at compile/assemble time.
- ▶ The compiler emits an **error** message or warning.
- ▶ Other times, the error goes unnoticed until something goes wrong at runtime.

Compiler warnings

- ▶ Aside from error, a compiler can help spot problematic code.
- ▶ Make compilers emit *warnings* when code is misleading, dangerous, . . .
- ▶ Use the compiler flag `-Wall` to turn on warnings in GCC.

Debug symbols

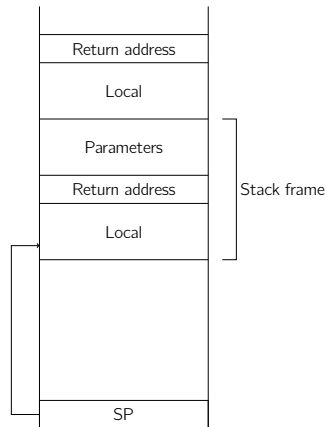
- ▶ During compilation, lots of information is lost.
- ▶ Source file and line, variable names, etc. all goes away.
- ▶ But can ask compiler to keep a lot of this information.
- ▶ Use the compiler flag `-g`.
- ▶ *Debug symbols* are added to the binary format.
- ▶ Debuggers can then use this information to help identify which instructions correspond to the source code.

DWARF

- ▶ DWARF is a standard debugging data format.
- ▶ Designed to work with ELF format.
- ▶ Contains information about every program variable, procedure, data type, ...
- ▶ E.g. Name and memory address (program counter) of each subroutine.

Unwinding the stack

- ▶ Recall: Stack frame.
- ▶ Program halts inside subroutine.
- ▶ Want to know how program got here (nested subroutines).
- ▶ Stack frame is unwound to find out.
- ▶ Use return address/stack pointer registers to find out which subroutine called this one.
- ▶ Function names stored in the debug info in the binary.
- ▶ Repeat process as necessary.



Debugging techniques

- ▶ Littering programs with print statements (e.g. `printf()`) can help find bugs.
- ▶ Unsophisticated, but can be useful.
- ▶ Often better to use a **debugger**.

Debuggers

- ▶ Debuggers are tools that trace the program execution.
- ▶ Keep track of the instructions and the results they execute.
- ▶ Allow you to inspect memory/variables.
- ▶ Mechanisms for stepping through programs one line at a time.
- ▶ Often can set *breakpoints* which define places to stop execution.

Kernel support for debuggers

- ▶ Debugger runs then launches the program in a separate process.
- ▶ Kernel provides library calls (`ptrace`) for debuggers to look inside another process.
- ▶ Can find register values, memory addresses, ...
- ▶ (Software) Interrupts are the key mechanism for communicating between the debugger and a process.

ISA software interrupts

Software Interrupts

An instruction that generates an interrupt.

For example:

- ▶ x86 INT X instruction: generate a software interrupt corresponding to interrupt vector X .
- ▶ x86 INT3 instruction: breakpoint instruction.
- ▶ Armv7-A SVC instruction: supervisor call (previously called SWI).
- ▶ Armv7-A BKPT: breakpoint instruction.

How a debugger uses software interrupts for breakpoints

Debugger

Set breakpoint
Run Debuggee
Wait for signal
Handle signal

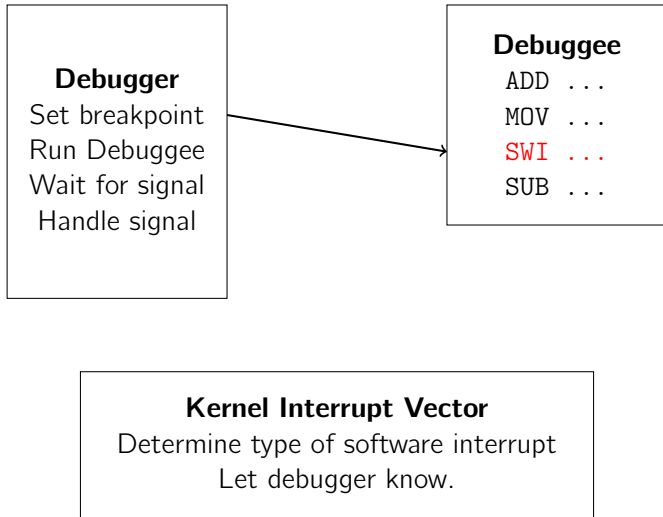
Debuggee

ADD ...
MOV ...
MOV ...
SUB ...

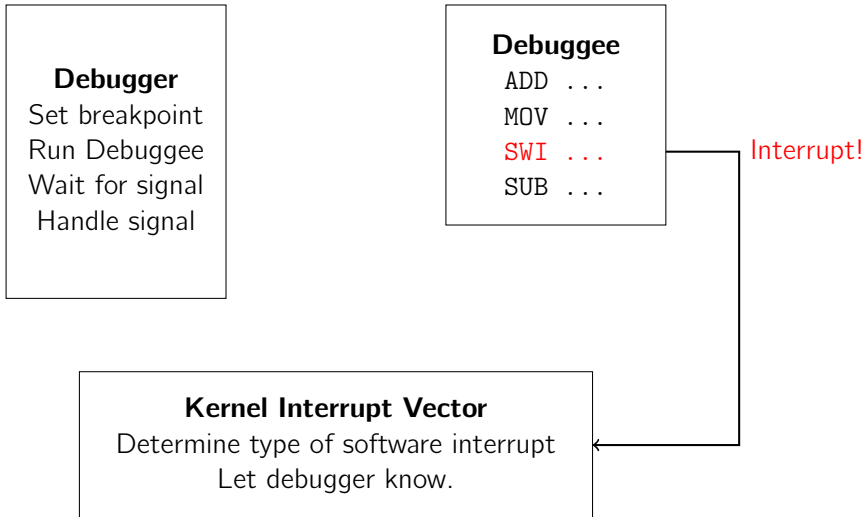
Kernel Interrupt Vector

Determine type of software interrupt
Let debugger know.

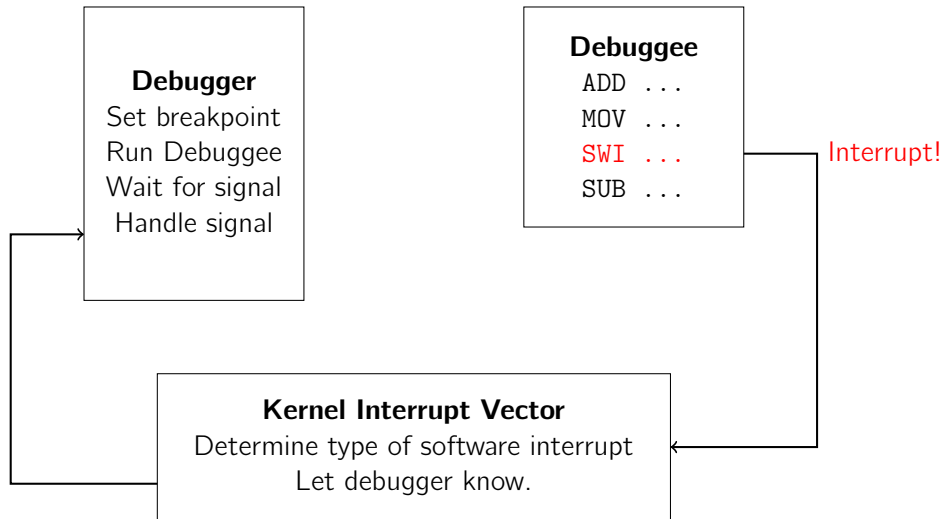
How a debugger uses software interrupts for breakpoints



How a debugger uses software interrupts for breakpoints



How a debugger uses software interrupts for breakpoints



How a debugger uses software interrupts for breakpoints

- ▶ Setting a breakpoints *replaces* the instruction with a software interrupt.
- ▶ Instructions are modified in place.
- ▶ Original instruction saved by the debugger for later execution.
- ▶ When execution reaches the breakpoint, it will interrupt, and return to the kernel via the debug interrupt vector.
- ▶ Kernel will then start the waiting debugger up again.
- ▶ When continuing, debugger restores program counter, instruction stream, register contents, ...

Hardware breakpoints

- ▶ Processors have a number of debug registers.
- ▶ Registers hold memory addresses on which to break.
- ▶ When get to particular address, processor generates interrupt signal.
- ▶ Useful for setting a break when data at memory address change.
- ▶ Status and control registers, to understand current state and when to break (read/write/execute).

Single step mode

- ▶ Recall: Status registers.
- ▶ Intel 8086 FLAGS register has a *trap flag*.
- ▶ This puts to processor in single step mode.
- ▶ Executes one instruction cycle then generates an interrupt.
- ▶ Could also set breakpoint at next instruction.

Apollo Guidance Computer

Apollo Program

- ▶ USA program where NASA landed first humans on the moon 1969 and brought them back to Earth, in 1969.
- ▶ Apollo 11 (Neil Armstrong, Michael Collins, Edwin (Buzz) Aldrin) mission that achieved that.
- ▶ Total of 17 Apollo missions.
- ▶ Lunar Module was the spacecraft used to get between lunar orbit and the moon surface.
- ▶ Contained the “Apollo Guidance Computer”, used to fly the Lunar Module.

AGU: Counters

- ▶ AGU had lots of counters collecting data from external sensors.
- ▶ E.g., acceleration would send a single bit to notify a change of a particular magnitude had occurred.
- ▶ Quicker than sending full readings, just track the change and accumulate in the AGU.
- ▶ Similar to interrupts, AGU checks after each instruction if count.
 - ▶ If so, accumulates the count rather than executing next instruction.
 - ▶ Taking only 11.7 microseconds per counter update.
- ▶ Unlike interrupts, no interrupt handler.
- ▶ General idea is called “cycle stealing”.

AGU: interrupts

- ▶ Also more standard interrupt routines which save processor state (registers).
- ▶ Used for:
 - ▶ timer interrupts (from autopilot and task scheduler (WAITLIST)),
 - ▶ DSKY (display keyboard),
 - ▶ radar,
 - ▶ processing of downlink telemetry back to ground control.
- ▶ These work using the usual interrupt vector and interrupt handler technique previously discussed.

Alarm 1201

- ▶ During Lunar Descent, 5 alarms during last 7 minutes.
- ▶ New jobs (programs) couldn't be scheduled on the AGU: AGU Software restarted 5 times.
- ▶ Previously when this alarm went off in a rehearsal (simulation), they aborted.
- ▶ Started then astronauts ran program querying range to landing site.

The problem: Radar Coupling Data Units (CDUs)

- ▶ The CDU power supply was not in phase, causing them to send many counters.
- ▶ Two CDUs, each sending 6,400 counter interrupts per second; total 12,800 interrupts/second and taking (cycle stealing) 15% of processing time.
- ▶ Processor ran programs as jobs, and once done, clears up resources (memory).
- ▶ Because of the counter interrupts, jobs didn't finish, so held their resources.
- ▶ Periodic jobs started failing due to lack of those resources, and caused a software restart of the AGU.

Further Reading

- ▶ <https://en.wikipedia.org/wiki/Debugger>
- ▶ <http://www.alexonlinux.com/how-debugger-works>
- ▶ <https://eli.thegreenplace.net/2011/01/23/how-debuggers-work-part-1>
- ▶ <https://eli.thegreenplace.net/2011/01/27/how-debuggers-work-part-2-breakpoints>
- ▶ <https://eli.thegreenplace.net/2011/02/07/how-debuggers-work-part-3-debugging-information/>
- ▶ https://en.wikipedia.org/wiki/Trap_flag
- ▶ <https://en.wikipedia.org/wiki/Ptrace>
- ▶ <https://en.wikipedia.org/wiki/DWARF>
- ▶ CppCon 2018: Simon Brand “How C++ Debuggers Work”:
<https://youtu.be/0DDrseUomfU>

Further Reading

- ▶ Frank O'Brien, The Apollo Guidance Computer: Architecture and Operation
- ▶ We're Go On That Alarm: Inside the Apollo Operating System
<https://medium.com/software-giant-leap/were-go-on-that-alarm-inside-the-apollo-operating-system-8d753e7a1e17>
- ▶ Apollo 11's "1202 Alarm" Explained, <https://www.discovermagazine.com/the-sciences/apollo-11s-1202-alarm-explained>
- ▶ Apollo 11 Lunar Surface Journal: Apollo 11 Program Alarms
<https://www.nasa.gov/history/alsj/a11/a11.1201-pa.html>